# PRESENCE: Server-Timed, Motion-Witnessed Proof of Human Process on Solana Mobile

with FACEDOWN, a phone-free commitment app built on it

Team WARRIO: Aaditya (iocryp15, lead) and nebula10

Technical whitepaper · 12 October 2026 · Solana devnet program `CFZsFPtv...FS2cT`

Source: https://github.com/Aaditya1273/FACEDOWN · Film: https://youtu.be/TL2G2HT28qY

**Abstract**

Commitment devices for phone use rely on things that cannot be checked: a local timer, a screenshot, or a human reviewer. Software can fake all three, and AI agents can now operate a phone. PRESENCE is a verification engine that turns "this person left their phone alone for $D$ minutes" into a record that is checked by a server and anchored on Solana. A session is a server-timed sequence of $N$ windows. Each window carries one presence check at a moment the user cannot predict. The user answers it by knocking on the back of the face-down phone. Each window seals a checkpoint into a SHA-256 hash chain signed by an ephemeral Android Keystore key bound into a Sign-In-With-Solana message. The checkpoint contains two accelerometer summaries: the share of the window the phone lay still and face-down ($\pi$), and the largest physical jolt felt within 300 ms of the answer ($\mu$). The ATTEST server re-derives the chain and evaluates a nine-clause predicate on its own clock. It then writes a create-once program-derived account and pays the reward in the same transaction. FACEDOWN, the first app, offers three modes: *Free* (sponsor-paid), *No-Loss* (the stake always comes back) and *Stake* (a quitter's stake is locked for 7/14/30 days, never taken). Nobody's money is ever paid to another user.

*Results.* We report measurements from a three-day devnet deployment on a real Android phone with the Solflare wallet and on the emulator: 56 sessions from 20 wallets, 19 settled and 9 rejected. The timing heuristic scored all 22 sessions that had enough answers at risk 0, including every session driven by `adb input tap`. Timing alone does not detect a script. The motion floor $\tau = 150\,\mathrm{mm/s^2}$ did: **29 of 30** injected taps on the real phone read 20–115 $\mathrm{mm/s^2}$ (median 35.5). One read 734 $\mathrm{mm/s^2}$. Every one of the eight injected-tap sessions contains at least one reading below the floor and would be rejected in production. The emulator's synthetic knocks (2,189–2,190 $\mathrm{mm/s^2}$) *pass*. This shows that a client able to inject sensor values defeats the motion check, and hardware attestation is required to close it. Re-deriving the chain and evaluating the policy takes 16 µs (median) for a 4-checkpoint session on a laptop CPU. The system comprises 2,046 lines of Python, 169 lines of Anchor Rust and about 5,000 lines of Kotlin, with 46 backend tests passing and 10 Android unit tests. Nothing here is on mainnet; all tokens are a valueless test mint.

**Keywords:** proof of human process, commitment devices, Solana Mobile, Seeker Genesis Token, accelerometer liveness, AI-agent resistance, hash-chained evidence, loss aversion without forfeiture

## 1 Introduction

Phones are built to be picked up. Blocking apps help, but they can be undone at the exact moment of the urge. *Commitment devices*, where you put something at stake on your own future behaviour, are among the best-studied tools in behavioural economics. They have a structural weakness on phones, though: someone must decide whether the commitment was kept. Today that someone is a local timer the user controls, a photo a human or model reviews, or the user's own word. None of these can back money. A timer can be fast-forwarded. A photo can be staged. A model reviewer costs money on every claim. There is also a newer problem. Agents that operate phone interfaces can now complete login and approval flows on their own [7]. Any reward that only checks for taps on a screen can be farmed by one.

We ask a narrow question: *can a phone produce evidence, strong enough to settle a financial commitment, that a person left it alone for a stated time and was physically there to answer when asked?* PRESENCE answers it with three design choices:

1. **The server keeps time.** Every duration and gap is measured on the ATTEST server's clock. Client timestamps are informational.
2. **The check-in is physical.** A presence check appears at a random moment in each window. The user answers it by knocking on the back of the face-down phone, and the accelerometer must feel the knock. An injected tap moves nothing.

3. **The verdict is anchored once.** A program-derived account (PDA) keyed by (profile, mission, day, sequence) can be created exactly once. The reward moves in the same transaction.

### 1.1 Contributions

- A **commitment calculus** (§5): a validated mission policy, a nine-clause server predicate evaluated on the server clock, a closed-form automation risk score, and two accelerometer statistics with exact definitions.
- A **hash-chained, key-bound evidence format** (§6). It is byte-identical in Kotlin and Python, shares golden vectors between them, and puts only 32 bytes on-chain.
- **Loss aversion without forfeiture** (§7). A cool-down lock replaces stake confiscation, so nobody ever wins anyone else's money. We give its expected cost next to forfeiture-based devices, plus pool accounting that cannot pay out more than sponsors deposited.
- An **Anchor program** (§8) that makes double-claiming structurally impossible and binds a profile to one Seeker Genesis Token.
- **Measurements from a live devnet deployment** with a real wallet (§9). They include a negative result: our timing heuristic alone does not detect a scripted client.
- A **limitations list ranked by severity** (§11), including the two attacks that defeat the current build.

## 2 Evidence Behind the Design

FACEDOWN's product decisions follow from published randomised and measurement studies. Table 1 maps each one to the mechanism it motivated. Every number is quoted from the paper's abstract.

**Table 1:** Published findings and the design decision each produced.

| Finding | Decision in FACEDOWN |
|---|---|
| Deposit contracts were accepted by 13.7% of smokers versus 90.0% for reward programs ($n = 2{,}538$). Among those who would accept either, deposits raised abstinence by 13.2 points [2]. | Free (sponsor-paid) first. No-Loss as the default way to put money in. Stake is opt-in for the committed. |
| Appointments more than doubled HIV-testing rates compared with financial commitment devices, and most takers of the latter lost their money [6]. | A daily *facedown appointment* with a reminder. Failure never costs principal. |
| Self-selected phone-reduction goals cut use by 26 min/day more (a 73% larger reduction) than assigned goals (RCT, $N = 149$, 12 weeks) [3]. | The user chooses the length (2, 10, 25, 60 min) and what is at stake. |
| A month-long RCT blocking mobile internet for two weeks improved sustained attention, mental health and well-being; 91% improved on at least one outcome [1]. | A *Facedown Hour*, not only short timers. A passive all-day ring (Presence Minutes). |
| A pause before opening a target app (the *one sec* self-nudge) reduced actual openings by 57% [4]. | *Urge Catcher*: the pause, plus an offer of a 10-minute verified session in place of the app. |
| Group-pooled incentives produced 3.2 kg more weight loss than individual incentives (95% CI 0.9–5.5) [5]. | *Squads* of 3–8 share one streak. |
| A motion-based signal distinguished human from agent-driven logins with 98.4% (table-top) and 92.6% (hand-held) accuracy over 116 pilot sessions [7]. zkSENSE reached 92% humanness accuracy from motion sensors [8]. | Knock check-ins and face-down stillness in every checkpoint. |
| In 9 of 12 NFT play-to-earn games, players who traded had a negative profit on average [9]. | No new token and no emissions. Rewards come only from sponsor deposits. |

## 3 Threat Model and Design Goals

**Principals.** The *user* runs the FACEDOWN Android app and holds a Solana wallet (Solflare over Mobile Wallet Adapter). The *ATTEST* server issues nonces, keeps time, verifies evidence, decides, and signs settlements with the attestor key. The *Presence program* on Solana records attestations and accepts writes only from the configured attestor. *Sponsors* deposit reward tokens into a mission pool. Solana *mainnet* Token-2022 state answers whether a wallet holds a Seeker Genesis Token (SGT).

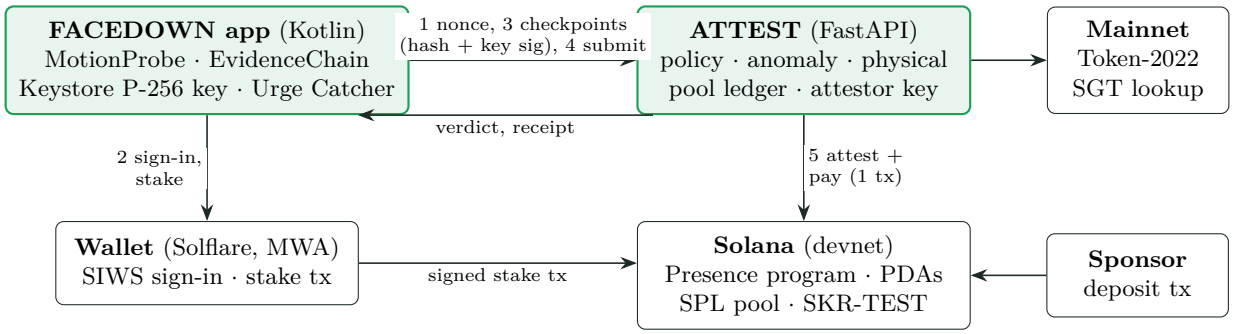



**Figure 1:** Architecture. The client collects evidence, ATTEST decides, and the chain anchors the result. The app never sends `verified`, `reward` or `assurance`. Raw sensor data never leaves the phone.

**Adversary.** The adversary is the user, or anyone acting for them, who wants a VERIFIED verdict (and its reward or returned stake) without leaving the phone alone. They control the phone's software. They can modify or re-sign the APK, replay traffic, run many wallets, drive the UI from `adb`, an accessibility service or an AI phone agent, and alter the stake transaction inside a wallet. They do not control the ATTEST server, its clock or the attestor key. They cannot forge ed25519 signatures or find SHA-256 collisions. In the version described here they *can* inject accelerometer values if they run a modified OS or an emulator (§11).

**Goals.**

**G1 Duration integrity.** A session cannot verify in less than its stated wall-clock time as measured by the server.

**G2 Continuity.** The app must stay in the foreground in every window. A missed window is visible.

**G3 Physical presence.** Each answer is accompanied by a physical jolt of the device, and the device lay face-down and still for most of each window.

**G4 Evidence integrity.** Checkpoints cannot be reordered, dropped, replayed into another session or edited after sealing.

**G5 Single claim.** A device's identity can claim a mission at most $c_{\max}$ times per UTC day. A claim's payout cannot be repeated.

**G6 Principal safety.** No verdict transfers one user's stake to another user.

**Non-goals.** We do not prove that the person is the wallet owner or one unique human (proof of personhood). We do not prove what the person was doing while away from the phone. We cannot stop the use of a second phone.

# 4 System Architecture

Figure 1 shows the components. The rule underneath them is that *authority sits with the party that cannot be patched by the adversary.* Table 2 lists who decides each fact.

## 4.1 Session lifecycle

A session moves through

$$\text{CREATED} \xrightarrow{\text{start}} \text{ACTIVE} \xrightarrow{\text{submit}} \text{SUBMITTED} \xrightarrow{\Pi=1} \text{VERIFIED} \xrightarrow{\text{tx confirmed}} \text{SETTLED},$$

$$\text{SUBMITTED} \xrightarrow{\Pi=0} \text{REJECTED}.$$

A replayed nonce ends in NONCE_REPLAY. A session is created when the server issues a nonce and a SIWS message. It becomes active when the wallet's signature over that message verifies, which also binds the ephemeral key (below). The server records the start time $t_s$ and the receipt time $r_i$ of every checkpoint on its own clock.

**Table 2:** Authority analysis: who decides each fact, and why not the client.

| Decision | Owner | Reason |
|---|---|---|
| Session nonce | ATTEST | Nonces chosen by the client could be replayed. |
| Wallet identity | ATTEST (ed25519 over the exact SIWS bytes it issued) | The client can be patched. |
| Seeker eligibility | ATTEST (mainnet Token-2022 state of the *signing* wallet) | An address alone proves nothing. |
| Duration and gaps | ATTEST server clock | Client timestamps can be forged. |
| Evidence validity | ATTEST recomputes every hash | — |
| PASS/FAIL | `policy.evaluate` on ATTEST | — |
| Claim limit, XP, streak, league | ATTEST database (streak mirrored on-chain) | — |
| Durable record | Presence program; only the attestor key may write | Anyone can inspect it. |

### 4.2 Why the server clock

Every focus timer on the market counts down on the device, so a modified client can finish a 60-minute session in a second. PRESENCE moves every time comparison to the server. A client can be *late*, which fails a session (§5), but it cannot be *early*. Clause $C_4$ below rejects any checkpoint that arrives sooner than the interval allows after the previous one. A session therefore cannot verify in less than $N(I-\delta)$ seconds of server time, however the client is modified.

## 5 The Commitment Calculus

### 5.1 Mission policy

A mission is a tuple $\mathcal{M} = (D, N, I, n_{\min}, a, c_{\max}, A_{\text{req}}, s, \beta, \mathsf{noloss}, \mathsf{cooldown}, \pi_{\min}, \tau)$. These are the duration, checkpoint count, interval, minimum answers per window, allowed misses, daily claim cap, required assurance level, stake, bonus rate, the two mode flags, the stillness floor (in percent) and the touch floor (in $\text{mm/s}^2$). `build_policy` refuses to load a mission unless

$$D = N \cdot I, \quad I \geq 5\,\text{s}, \quad 0 \leq a < N, \quad 1 \leq c_{\max} \leq 255, \quad 0 \leq \beta \leq 1, \quad 0 \leq \pi_{\min} \leq 100, \quad \tau \geq 0, \tag{1}$$

$$\mathsf{cooldown} \Rightarrow s > 0 \wedge \neg\mathsf{noloss}, \qquad \mathsf{noloss} \Rightarrow s > 0 \wedge \beta = 0. \tag{2}$$

The bound $c_{\max} \leq 255$ exists because the on-chain claim index is a `u8`. The rule $\mathsf{noloss} \Rightarrow \beta = 0$ holds because a bonus in No-Loss would have no funding source: nobody forfeits anything. Policies that require witnesses (P3) are refused at load time, because P3 is not implemented. The deployed missions use $\pi_{\min} = 60\%$, $\tau = 150\,\text{mm/s}^2$, $a = 1$ and $n_{\min} = 1$. The windows are $4 \times 30\,\text{s}$ (quick), $2 \times 300\,\text{s}$ (urge break), $5 \times 300\,\text{s}$ (25 min) and $4 \times 900\,\text{s}$ (Facedown Hour).

### 5.2 Unpredictable check-ins

Window $w \in \{0, \ldots, N-1\}$ spans $[wI, (w+1)I)$. Its presence check appears at the offset

$$\theta_w = I\,(0.15 + 0.45\,U_w), \qquad U_w \sim \mathcal{U}[0,1) \text{ i.i.d.}, \tag{3}$$

so it lands somewhere in the 15–60% band of the window (Figure 2). The phone buzzes, and the user knocks. The answer time $\lambda_w$ is measured from when the check appears to when it is answered (or $-1$ if it never is). The motion probe ignores the buzz itself (a quiet period around the phone's own vibration) so the motor is not mistaken for a knock. The randomness is drawn on the client, so it stops a person from anticipating the check, not a modified client (§11).

### 5.3 Physical evidence

`MotionProbe` samples the accelerometer $\mathbf{a}_t = (x_t, y_t, z_t)$ at about 100 Hz, which is below the 200 Hz rate that requires a permission. It keeps a 400-sample ring buffer (about 4 s). With $m_t = \|\mathbf{a}_t\|$ it maintains a slow

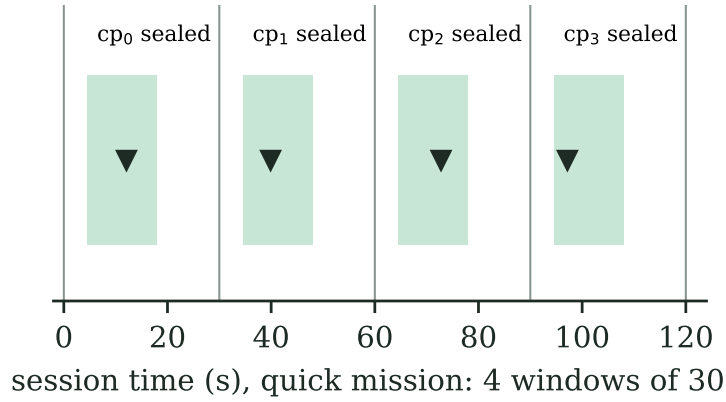



**Figure 2:** Check-in timing for a quick mission. Shaded areas are the 15–60% band where a check can appear, and triangles are one sample of $\theta_w$. A checkpoint is sealed and sent at each window boundary.

baseline and a face-down filter:

$$b_t = b_{t-1} + 0.02\,(m_t - b_{t-1}), \qquad \bar{z}_t = \bar{z}_{t-1} + 0.05\,(z_t - \bar{z}_{t-1}), \qquad \mathsf{faceDown}_t \iff \bar{z}_t < -7\ \mathrm{m/s^2}. \tag{4}$$

*Knock.* A knock is registered at sample $t$ if

$$|m_t - b_{t-1}| > \kappa \ \wedge\ t > t_{\text{quiet}} \ \wedge\ t - t_{\text{last knock}} > 350\,\text{ms}. \tag{5}$$

The threshold is personal. Calibration listens at $0.15\,\mathrm{m/s^2}$ for three knocks $k_1, k_2, k_3$ and sets

$$\kappa = \max\!\big(0.2,\ \tfrac{1}{2}\,\text{median}(k_1, k_2, k_3)\big)\ \mathrm{m/s^2}, \tag{6}$$

with a default of $\kappa = 0.35\,\mathrm{m/s^2}$. Sensor noise on our test phone measured $\le 0.09\,\mathrm{m/s^2}$.

*Touch motion $\mu$.* For an on-screen tap at time $t_0$, the baseline is the median of $m$ over $[t_0 - 1.5\,\text{s},\ t_0 - 150\,\text{ms})$, and

$$\mu(t_0) = 1000 \cdot \max_{t \in [t_0 - 150\,\text{ms},\ t_0 + 300\,\text{ms}]} \big|m_t - \text{median}\big(m_{[t_0 - 1.5\,\text{s},\ t_0 - 150\,\text{ms})}\big)\big| \quad [\mathrm{mm/s^2}]. \tag{7}$$

A knock contributes its own deviation $1000\,|m_t - b_{t-1}|$. Each window reports $\mu_w$ = max over its answers, or $-1$ if there were none. A finger moves the phone. An `adb input tap`, an accessibility gesture or an agent's injected event does not.

*Stillness $\pi$.* Time is cut into one-second buckets $B$. A bucket *counts* if it has at least 5 samples and starts more than 2 s after the last answer, so the knock itself is never held against the user. A counted bucket is *still* if

$$\text{sd}_{t \in B}(m_t) < 0.08\ \mathrm{m/s^2} \ \wedge\ \text{mean}_{t \in B}(z_t) < -7\ \mathrm{m/s^2}, \qquad \pi_w = \Big\lfloor 100\,\frac{|\{B \text{ still}\}|}{|\{B \text{ counted}\}|} \Big\rfloor. \tag{8}$$

## 5.4 The server predicate

Let $t_s$ be the start, $t_u$ the submit time, and $r_0 < \cdots < r_{K-1}$ the checkpoint receipt times on the server clock, with $r_{-1} = t_s$. Each checkpoint $i$ carries the foreground flag $f_i$, the answer count $n_i$, the answer time $\lambda_i$ (in ms), $\pi_i$ and $\mu_i$. With tolerance $\delta = 3\,\text{s}$ and achieved assurance $A$, the session passes the policy stage iff

$$\begin{aligned} &C_1 : A \ge A_{\text{req}} && C_2 : K \ge N && C_3 : t_u - t_s \ge D - \delta \\ &C_4 : \forall i :\ r_i - r_{i-1} \ge I - \delta && C_5 : \forall i :\ f_i = 1 && C_6 : \big|\{i : n_i < n_{\min}\}\big| \le a. \end{aligned} \tag{9}$$

Then three further stages apply, in order:

$$C_7 :\ \varrho(\boldsymbol{\lambda}) < 0.9 \qquad \text{(automation risk, §5.5)} \tag{10}$$

$$C_8 :\ \forall i :\ n_i > 0 \Rightarrow \mu_i \ge \tau \qquad \text{(human touch)} \tag{11}$$

$$C_9 :\ \forall i :\ \pi_i \ge \pi_{\min} \qquad \text{(phone down)} \tag{12}$$

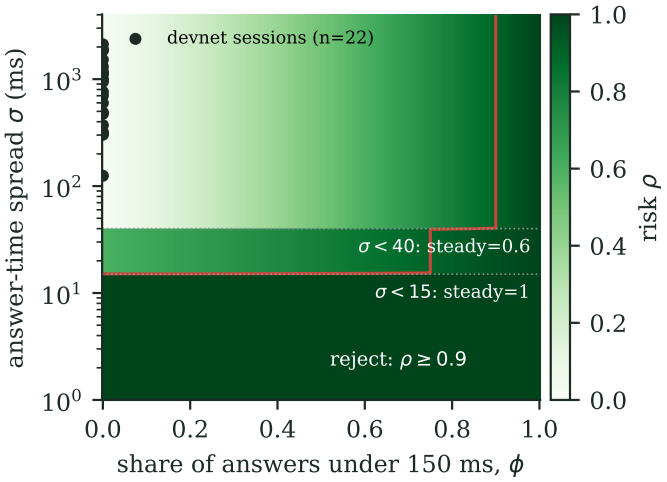



**Figure 3:** Automation risk $\varrho(\phi, \sigma)$ from Eq. (13). The red contour is the rejection boundary $\varrho = 0.9$. Dots are the 22 devnet sessions with at least three answers. All have $\phi = 0$ and $\sigma \geq 125\,\text{ms}$, so all score $\varrho = 0$, including the sessions driven by `adb`.

The verdict is

$$\text{verdict} = \begin{cases} \text{POLICY__NOT__SATISFIED} & \text{if } \neg(C_1 \wedge \cdots \wedge C_6) \\ \text{AUTOMATION__DETECTED} & \text{else if } \neg C_7 \vee \neg C_8 \\ \text{POLICY__NOT__SATISFIED} & \text{else if } \neg C_9 \\ \text{VERIFIED} & \text{otherwise } (\Pi = \bigwedge_{j=1}^{9} C_j) \end{cases}$$

Every violated clause in $C_1..C_6$ is listed in the receipt, not just the first. In *development mode* (emulators, which have no real motion), $C_8$ and $C_9$ are evaluated and reported as `FLAGGED` rather than failing the session. Every receipt carries `"mode": "development"`. Development mode is never the default, and a test enforces this.

Two consequences follow directly. From $C_3$ and $C_4$, the minimum server time to a verdict is $\max(D - \delta,\, N(I - \delta))$, which is 117 s for the quick mission. From $C_5$, opening any other app in any window fails the session, regardless of the miss allowance $a$.

## 5.5 Automation risk

Let $R = \{\lambda_i : \lambda_i \geq 0\}$ be the answered checks. If $|R| < 3$, then $\varrho = 0$. Otherwise

$$\phi = \frac{|\{\lambda \in R : \lambda < 150\}|}{|R|}, \qquad s(\sigma) = \begin{cases} 1 & \sigma < 15 \\ 0.6 & 15 \leq \sigma < 40 \\ 0 & \sigma \geq 40 \end{cases}, \qquad \varrho = 1 - (1 - \phi)\,\big(1 - s(\sigma_R)\big), \tag{13}$$

where $\sigma_R$ is the population standard deviation of $R$ in ms. The 150 ms floor comes from simple reaction time to an unexpected stimulus. The two factors combine like independent detectors (a noisy-OR). The rejection region $\varrho \geq 0.9$ has a closed form:

$$\varrho \geq 0.9 \iff \sigma < 15 \quad \vee \quad (15 \leq \sigma < 40 \wedge \phi \geq 0.75) \quad \vee \quad \phi \geq 0.9. \tag{14}$$

Figure 3 plots the surface with the deployment's sessions on it. This is a heuristic, not a model. It catches instant or metronome-steady answers. A script that adds human-like random delays defeats it, and our own measurements show this (§9).

# 6 Evidence Chain and Key Binding

Let $\|$ denote joining with the byte `0x7C` ('|'), and let $H$ be SHA-256. With server nonce $\eta$ and session id *sid*:

$$h_{-1} = H(\texttt{"PRESENCE/genesis/v1"} \,\|\, sid \,\|\, \eta) \tag{15}$$

$$p_i = \texttt{"}i|t_i^{\mathrm{cl}}|e_i|f_i|n_i|\lambda_i|\pi_i|\mu_i\texttt{"} \tag{16}$$

$$h_i = H(\texttt{"PRESENCE/cp/v3"} \,\|\, h_{i-1} \,\|\, p_i \,\|\, \eta) \tag{17}$$

$$root = H(\texttt{"PRESENCE/root/v1"} \,\|\, h_{K-1} \,\|\, 0^{32}) \tag{18}$$

The 32 zero bytes are the witness root (P3 not implemented). Here $t_i^{\mathrm{cl}}$ and $e_i$ are the client wall-clock and elapsed times, which are *informational only*. The domain-separation strings stop a checkpoint hash from being reused as a genesis or root. Including $\eta$ in every link binds the chain to one session.

**Key binding.** At session start the app generates an ephemeral P-256 key in the Android Keystore. Its public key is embedded in the SIWS message that the wallet signs. Each checkpoint is sent as $(p_i, h_i, \mathrm{Sig}_k(h_i))$. ATTEST verifies the wallet's ed25519 signature over the exact message bytes it issued. It then recomputes $h_i$ from $h_{i-1}$ and $p_i$, verifies the P-256 signature, and stores the chain head. A checkpoint from another session, from a reordered chain or with an edited field fails one of these three checks. The key is destroyed when the session ends.

**Privacy.** Each window discloses one foreground bit, an answer count, one answer time and two motion summaries. Raw sensor samples stay on the device. There is no location, camera, microphone or contacts access. Only the 32-byte root and the assurance level reach the chain.

**Cross-language determinism.** `evidence.py` and `EvidenceChain.kt` share golden vectors, so an independent verifier, such as a future Guardian (§13), can re-derive any root from the receipt.

# 7 Settlement and Economics

## 7.1 Three modes

Let $s$ be the stake, $\beta$ the bonus rate, $w$ the sponsor reward, and F/Q mean finish/quit.

**Table 3:** Payoffs by mode. No cell moves one user's money to another user.

| | **Free** | **No-Loss** | **Stake (Cool-down)** |
|---|---|---|---|
| At risk | nothing | $s$ locked for the session | $s$ |
| Finish (F) | $+w$ (sponsor) | $s$ back $+\,w$ | $s$ back $+\min(\beta s,\ P - R)$ |
| Quit (Q) | 0 | $s$ back (refund) | $s$ locked $L(k)$ days, then back |
| Abandoned | 0 | refunded after expiry | as quit |

## 7.2 Pool accounting

Each mission has a pool with verified sponsor deposits $D_p$ (credited only after ATTEST confirms the deposit transaction on-chain), forfeits $F_p$ (zero under current rules) and outstanding reservations $R_p$:

$$P_{\mathrm{rem}} = D_p + F_p - R_p. \tag{19}$$

A sponsor reward $w$ is reserved at verification only if $P_{\mathrm{rem}} \geq w$. A Stake bonus is

$$bonus = \min\big(\,\mathrm{round}(s\beta),\ \max(P_{\mathrm{rem}}, 0)\big), \tag{20}$$

and the app shows it live before the user commits. By construction, the sum of all payouts never exceeds deposits, and the operator never needs to mint (Figure 5, left). Rewards are SPL *transfers* from the pool. Pointing `REWARD_MINT` at real SKR (`SKRbvo6G...ZhW3`) is the only change needed for mainnet.

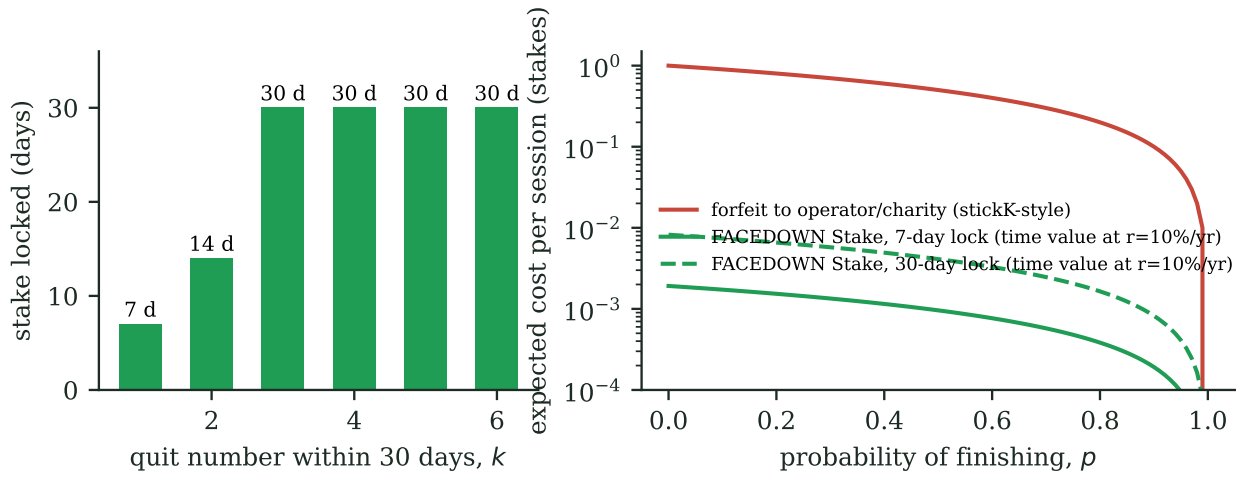



**Figure 4:** Left: lock length $L(k)$ from Eq. (21). Right: expected money cost per session against the probability of finishing (log scale). The forfeiture curve is a stickK-style commitment contract. FACEDOWN's Stake cost is the time value of the locked stake at an illustrative $r = 10\%/\text{yr}$. No-Loss and Free cost zero (not drawn on a log axis).

### 7.3 Cool-down Lock: loss aversion without forfeiture

Let $k$ be the number of cool-down locks a wallet has taken in the last 30 days, including this one. The lock length is

$$L(k) = \big(7,\ 14,\ 30\big)_{\min(k,3)} \text{ days.} \tag{21}$$

A quitter's stake is never taken. The cost of quitting is the time value of the locked stake. With finishing probability $p$ and opportunity rate $r$:

$$\mathbb{E}[C_{\text{forfeit}}] = (1-p)\,s, \qquad \mathbb{E}[C_{\text{cool-down}}] = (1-p)\,s\,\frac{r\,L(k)}{365}, \qquad \frac{\mathbb{E}[C_{\text{cool-down}}]}{\mathbb{E}[C_{\text{forfeit}}]} = \frac{r\,L(k)}{365}. \tag{22}$$

At $r = 10\%/\text{yr}$ the ratio is 0.19% for a 7-day lock and 0.82% for a 30-day lock (Figure 4). The money cost is therefore close to zero. The psychological cost, not having access to your money, is what remains. This trades commitment strength for safety on purpose. Halpern et al. found deposits more effective among people who would accept them, but almost nobody accepted them [2]. Derksen et al. found most commitment-device takers lost their money [6]. Whether a lock is *enough* commitment is an open empirical question we have not measured.

### 7.4 Streaks, freezes and squads

Let $\ell$ be the user's last verified UTC day and $\phi_f$ the freezes they hold. A verification on day $d$ updates the streak $S$ as

$$m = d - \ell - 1, \qquad S' = \begin{cases} S+1, & m \le \phi_f \quad \text{(consumes } m \text{ freezes)} \\ 1, & \text{otherwise,} \end{cases} \tag{23}$$

and the streak is displayed as live while $d_{\text{today}} - \ell \le 1 + \phi_f$. A freeze costs 2 SKR: half is burned with `BurnChecked` and half is protocol revenue. It is the only token sink, and it exists so that holding SKR has a use. A *squad* of $n \le 8$ members has one streak, which counts the consecutive days on which *every* member who had joined by then verified. If each member verifies independently with probability $q$, a squad day succeeds with probability $q^n$. The chance that a streak survives $d$ days with $f$ freezes is

$$\Pr[\text{alive at } d] = \sum_{j=0}^{f} \binom{d}{j} (1-q^n)^j\,(q^n)^{d-j}. \tag{24}$$

Figure 5 (right) shows the trade-off. A squad of three at $q = 0.9$ loses its streak within about two weeks. At $q = 0.97$ it lasts as long as a solo user at $q = 0.9$. Squads multiply social pressure and also fragility, which is why membership is capped at 8 and freezes exist. Eq. (24) is a model (independent days, freezes not replenished), not a measurement.

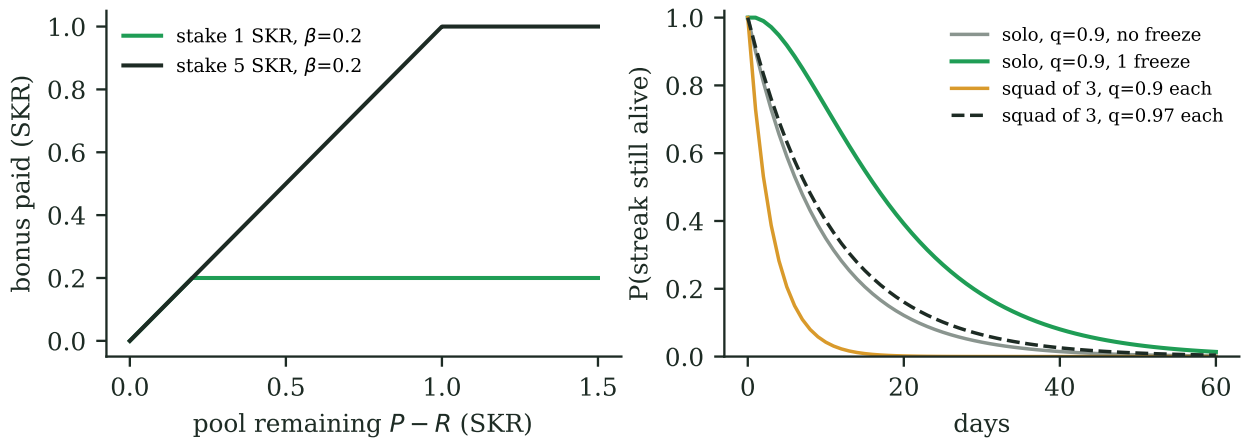



**Figure 5:** Left: Stake bonus from Eq. (20) as the pool drains. The 1-SKR stake saturates at its 0.2 cap; the 5-SKR stake is limited by the pool up to 1.0. Right: modelled streak survival from Eq. (24).

### 7.5 Stake integrity

ATTEST builds the stake transaction $T$ (an SPL transfer of $s$ to the pool) and the wallet signs it. Wallets legitimately add instructions: Solflare prepends priority-fee instructions. ATTEST accepts the signed $T'$ only if

$$\mathrm{ix}(T') \setminus \{\text{ComputeBudget}\} = \mathrm{ix}(T), \tag{25}$$

with the same fee payer. A changed amount, recipient or payer is refused, and unit tests cover each case.

### 7.6 Why no token and no rake on losers

Play- and move-to-earn economies pay rewards from emissions and collapse when new deposits slow [9]. FACEDOWN has no token. Its rewards are sponsor transfers: parents funding a student, employers funding focus hours, or protocols funding campaigns. India's Promotion and Regulation of Online Gaming Act (2025) prohibits online money games. A design where stakers win quitters' stakes risks falling within that definition. Under FACEDOWN's rules (No-Loss refunds, Stake locks then returns, sponsor-only rewards) no user's money is ever paid to another. That is our reading of the law, not legal advice, and it needs review before a public launch.

## 8 On-Chain Anchoring

The Presence program (169 lines of Anchor) has two account types and one write path. `record_attestation(day, mission_id, root, A, sgt, seq)` creates

$$att = \mathrm{PDA}\big(\texttt{"attestation"},\ profile,\ H(mission_id),\ day_{\mathrm{le}},\ seq\big) \tag{26}$$

with Anchor's `init`, which fails if the account exists. A claim index $seq < c_{\max}$ that has already been used therefore cannot be used again, by anyone, ever (G5). The instruction also requires:

- the signer is `Config.attestor` (`has_one`);
- $A \in \{1, 2, 3, 4\}$;
- $day \in \{\lfloor t/86400 \rfloor,\ \lfloor t/86400 \rfloor - 1\}$ on the cluster clock;
- the profile binds the SGT mint on first use, and every later attestation must present the same mint (`SgtMismatch`).

The program mirrors the streak with $\Delta = day - \ell$: $\Delta = 0$ keeps it, $\Delta = 1$ adds one, and anything else resets it to 1. It adds $A$ to reputation. The attestor pays rent, so users never need SOL to be attested. The reward or stake return is included *in the same transaction* as `record_attestation`. If the PDA already exists, the payout fails with it.

**Identity.** P1 needs a SIWS signature over a fresh server nonce *and* an SGT held by the signing wallet. The SGT is a Token-2022 mint whose mint authority is `GT2zuH...` and whose group is `GT22s8...`, read from mainnet.

ATTEST keys claim limits by the SGT mint, not the wallet. One Seeker can claim a sponsored mission $c_{\max}$ times per day however many wallets it creates. In development mode the key is `dev:<wallet>` and the receipt says `DEV_BYPASS`.

**Integration.** Third parties (airdrop issuers, quest platforms) can read `DailyAttestation` accounts directly, or call `verifyPresence()`, to gate rewards on proof of a human process. An example: "5 verified facedown sessions" is a quest that a bot farm cannot complete without the motion signal.

# 9 Implementation and Evaluation

## 9.1 Implementation

**Table 4:** Implementation (line counts from the working tree on 12 Oct 2026).

| Component | Stack | Responsibility | Lines |
|---|---|---|---|
| FACEDOWN app | Kotlin, Jetpack Compose | UI, MotionProbe, EvidenceChain, MWA, Urge Catcher | ≈5,000 |
| ATTEST | Python, FastAPI, SQLite | policy, anomaly, identity, pools, settlement | 2,046 |
| Presence program | Rust, Anchor | config, profile, create-once attestation | 169 |
| Tests | pytest; JUnit | 46 pass, 2 skipped; 10 (chain 2, motion 7, minutes 1) | — |

## 9.2 Deployment

ATTEST ran in development mode against Solana devnet from 9 to 12 October 2026. Clients were a realme RMX3242 (Android 13) with Solflare, and the Android emulator with Solana Mobile's test wallet. The real phone was connected over USB with `adb reverse`; the server was not publicly deployed. Real-phone sessions were driven by `scripts/device_e2e.py`, which answers check-ins with `adb input tap`. This makes them a direct measurement of an *automated* client on real hardware. Emulator sessions drove knocks and face-down through the virtual accelerometer.

**Table 5:** Deployment totals (from `.devnet/attest.db`, regenerated by `make_figures.py`).

| | | | |
|---|---|---|---|
| Sessions / wallets | 56 / 20 | Checkpoints sealed / answered | 119 / 95 |
| Settled / rejected | 19 / 9 | Rejected: left foreground / missed checks | 7 / 2 |
| Active (abandoned) / created only | 3 / 25 | Stakes returned / locked / issued unsigned | 10 / 7 / 13 |
| Sponsor deposits verified | 11 | Streak-freeze orders | 3 |
| Squads / members | 5 / 7 | Claims recorded | 19 |

One stake row is recorded as FORFEITED. It comes from an earlier build whose rules forfeited a quitter's stake, before the Cool-down Lock replaced forfeiture. The 25 CREATED sessions were issued a nonce whose sign-in the wallet never completed. Real-phone transactions are listed in Table 8.

## 9.3 Result 1: timing does not detect a script; motion does

All 22 sessions with $\geq 3$ answers scored $\varrho = 0$ (Figure 3). This includes the eight real-phone sessions answered by `adb input tap`. Their answer times inherit the latency of the script's UI polling (`uiautomator dump`, then tap) and so look human: none is under 150 ms, and $\sigma \geq 125$ ms. The motion statistic separated them. The 30 injected taps read $\mu \in [20, 734]\,\mathrm{mm/s^2}$, median 35.5, and 29 were below $\tau = 150$. Every one of the eight sessions contains a sub-floor answer, so $C_8$ fails for all eight. In development mode, 11 settled receipts carry `human_touch: FLAGGED` and would have been AUTOMATION_DETECTED in production. The single 734 reading is unexplained. Our working hypothesis is a coincident physical bump to the phone or cable. It shows that one reading can clear the floor by chance. The rule requires *every* answered window to clear it, so a session-level miss needs every reading to be lucky.

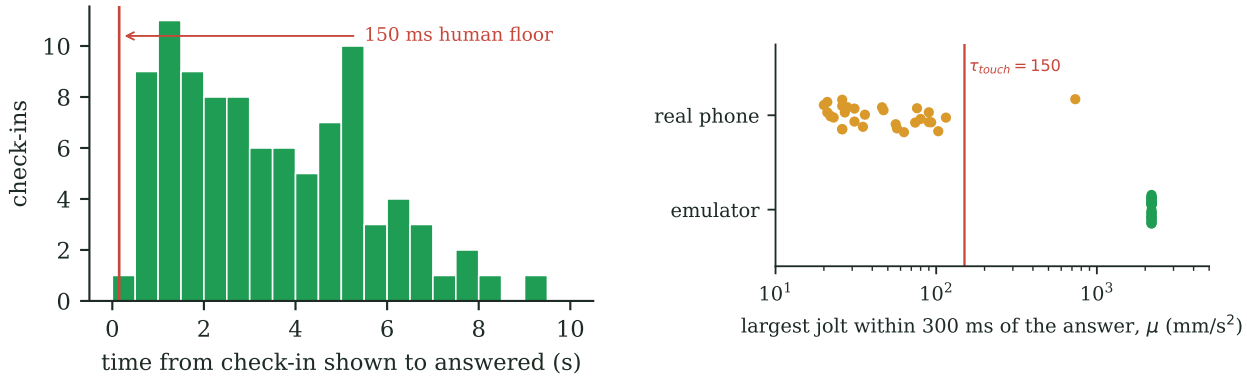



**Figure 6:** Left: the 95 answer times (median 3.07 s, min 344 ms). None is under the 150 ms floor, so $\phi = 0$ everywhere. Right: touch motion $\mu$ by source. Injected taps on the real phone fall below $\tau$ in 29 of 30 cases. The emulator's synthetic knocks clear it easily.

## 9.4 Result 2: injected sensors defeat the motion check

All five settled sessions whose receipts show both `human_touch` and `phone_down` as `PASSED` came from the emulator. Its virtual accelerometer produced knocks of 2,189–2,190 mm/s$^2$ (about 15× the floor) and stillness of 84–100%. To the server these are indistinguishable from a person. Any client that can write accelerometer values (an emulator, a rooted phone, a hooked sensor HAL) passes $C_8$ and $C_9$. This is the strongest attack on the current build. Only hardware-backed attestation that the app runs unmodified on a genuine device closes it (P4, §13).

## 9.5 Result 3: why sessions failed

Of the 9 rejections, 7 were caused by leaving the foreground ($C_5$) and 2 by missed check-ins beyond $a$ ($C_6$). None failed on timing ($C_3$, $C_4$): no client in the deployment attempted to compress time.

## 9.6 Verifier cost

Table 6 reports the time to recompute the chain and root (Eqs. (15)–(18)) and evaluate the full predicate (clauses $C_1$–$C_9$), over 2,000 runs on an Intel i5-13420H with CPython 3.14. Cost is linear in $K$. Repeated runs vary by about 10%; the table is the run saved in `measured.json`. Verification is negligible next to Solana confirmation (seconds), so the engine's throughput is bounded by RPC and database I/O, not cryptography.

**Table 6:** Chain recompute and policy evaluation latency (measured).

| Checkpoints $K$ | p50 (µs) | p99 (µs) |
|---:|---:|---:|
| 4 | 16.4 | 37.9 |
| 16 | 31.4 | 56.3 |
| 64 | 85.8 | 191.8 |
| 256 | 259.5 | 437.3 |

## 9.7 What a cheater would try

Table 7 lists every attack we considered and how far each defence has been exercised.

**Table 7:** Attacks and their status. "Reproduced" means observed in the deployment; "unit-tested" means covered by the test suite.

| Attempt | Why it fails | Status |
|---|---|---|
| Fast-forward the timer | $C_3$, $C_4$ on the server clock | unit-tested |
| Use the phone mid-session | $C_5$ foreground bit; $C_9$ stillness | reproduced (7 rejections) |
| Script / AI agent answers | $C_8$: injected input has no jolt | reproduced (30 taps, 8 sessions) |
| Replay or forge evidence | nonce in every link; P-256 sig per checkpoint | unit-tested |
| Many wallets, one device | claims keyed by SGT mint | unit-tested (not on a Seeker) |
| Swap the stake tx in the wallet | instruction-set equality modulo ComputeBudget | unit-tested |
| Claim the payout twice | create-once PDA, payout in same tx | by construction |
| **Inject sensor values** | **not defended (needs P4)** | **reproduced (emulator)** |
| Second phone | not defended (non-goal) | — |

**Table 8:** Real-phone (realme, Android 13) + Solflare transactions on devnet, current build.

| | |
|---|---|
| Free session: attestation + 0.1 reward | `65dmpG1S...bC9XsVCe` |
| No-Loss: lock 1 SKR-TEST / finish, stake back + reward | `htM37NsU...MUBtjrBd` / `3tLytYKx...sH1DSLMo` |
| Stake: 1 SKR-TEST / finish, attestation + stake back | `3CwXnpdq...2dRPfNB5` / `RGkdQhLm...6jsWguRZ` |
| Stake, left the app: Cool-down Lock to 19 Oct | `3XhHZQcf...cxoS3G5b` |
| Streak freeze: paid / half burned | `4CsvYY3p...L8iYGXxp` / `pcoHdAfv...GE1HCz7n` |

# 10 Findings from Testing Against a Real Wallet

1. **Solflare rotates the MWA auth token on every approval.** Reusing the first token after an app restart failed with "authorization request failed". Every wallet call now saves the newest token and retries once.
2. **Wallets add instructions.** Solflare prepends priority-fee instructions before signing. A strict "bytes-equal" check would reject every honest stake. The rule became equality modulo ComputeBudget.
3. **Solflare sometimes described a devnet stake as "for mainnet".** Building stakes on a *finalized* blockhash made it stop. Our reading is that its RPC had not seen a just-confirmed hash. That is not confirmed.
4. **Response-time statistics cannot see scripted UI drivers.** See Result 1. The heuristic stays as a cheap filter for naive bots. The motion floor carries the real weight.
5. **Emulators stop delivering sensor events when the screen is off.** Presence Minutes (the passive face-down ring) and Urge Catcher were therefore verified only on the emulator with the screen on. Screen-off sampling on real phones still needs checking.
6. **OEMs differ.** realme blocks enabling an accessibility service over `adb`, so Urge Catcher must be switched on by hand in Settings.

# 11 Trust Assumptions and Limitations

Ordered by severity.

1. **Sensor injection.** A modified OS, an emulator or a hooked sensor HAL can produce $\pi$ and $\mu$ that pass. Reproduced. The motion check stops UI-level automation (scripts, accessibility, AI agents), not a modified device.
2. **Single operator.** One ATTEST server holds the attestor key, and `Config.attestor` is one public key. A compromised or dishonest operator can attest anything. Mitigation is a Guardian quorum (§13), which is designed but deliberately has no code stub.
3. **Client-side randomness.** $\theta_w$ in Eq. (3) is drawn on the phone. A modified client knows when checks will appear. This matters only together with item 1, because answers still need a physical jolt.
4. **Real knocks not yet measured on a real phone.** Knock detection and face-down were verified on the emulator's virtual sensor. Real-phone runs used injected taps. A calibrated table of real knocks across phones and surfaces is the next measurement. Until it exists, the $150\,\mathrm{mm/s^2}$ floor is a design value. All we know is that injected taps on a real phone read well below it (20–$115\,\mathrm{mm/s^2}$, one outlier at 734) and synthetic knocks well above it.

5. **SGT not tested on a Seeker.** The verifier is implemented and unit-tested. Every receipt in this deployment is `DEV_BYPASS`.
6. **This phone, not this person.** We prove a device lay face-down with someone answering it, not who that was or that they had no other phone. In No-Loss, cheating only cheats yourself. In sponsored pools the SGT limits each device to $c_{\max}$ claims.
7. **Behavioural effect unmeasured.** We cite trials of related interventions, but we have not run one. The benefit of FACEDOWN itself is unproven.
8. **Legal.** The design is meant to stay outside India's online money game definition. That is untested and needs counsel.
9. **Test tokens only.** Everything ran on devnet with SKR-TEST and a development-only faucet, which the server refuses to run on mainnet.

# 12 Related Work

**Commitment devices.** Deposit contracts work for those who take them up, but uptake is low [2]. Appointments achieve commitment without loss [6], and group pools amplify incentives [5]. Commercial commitment-contract services verify with referees or self-report and send forfeits to a charity or the operator. PRESENCE replaces the referee with server-checked sensor evidence and removes the forfeit. **Digital well-being.** Focus timers that grow a virtual tree on the device, and app blockers, rely on local state. The one-sec nudge shows that a pause reduces openings [4]. Blocking mobile internet improves attention [1], and self-chosen goals outperform assigned ones [3]. **Humanness from sensors.** zkSENSE classifies motion during interaction and wraps the result in a zero-knowledge proof [8]. Liu and Huang show agent-driven MFA completing end to end, and a motion signal that separates it from humans [7]. PRESENCE uses a deliberately simpler, inspectable statistic, and places it inside a signed, hash-chained process record with an economic outcome. That record is what makes the signal settle money rather than only score risk. **Token incentive economies.** Emission-funded play-to-earn games left most trading players at a loss [9]. That is the reason for sponsor-only rewards.

# 13 Future Work

1. **P4 hardware attestation.** Require Android Key Attestation for the ephemeral key, and a platform integrity verdict, as a policy clause. This closes limitation 1.
2. **Guardian quorum.** Extend `Config` to a set $G$ of Guardian keys and a threshold $m$. Each Guardian runs its own ATTEST over the same evidence (the chain is deterministic) and signs $M_g$ below. `record_attestation` then accepts iff

$$M_g = (\mathit{profile}, \mathit{mission}, \mathit{day}, \mathit{seq}, \mathit{root}), \qquad \left|\{g \in G : \text{Ed25519.Verify}(pk_g, M_g, \sigma_g)\}\right| \geq m,$$

   checked through the Ed25519 precompile. Guardians stake SKR behind their signatures and can be slashed.
3. **P3 witnesses.** Fresh-challenge BLE co-presence witnesses fill the reserved 32-byte witness root.
4. **Server-issued challenges.** Move $\theta_w$ to the server: it reveals each window's check-in time only at that moment. This closes limitation 3.
5. **A learned presence model.** Replace the hand-set thresholds with an on-device classifier trained on opt-in real traces. We have not built it yet because we have no data to train it on.
6. **Yield-funded No-Loss.** On mainnet, locked stakes earn lending yield that funds the finisher reward in place of sponsors.
7. **A randomised trial.** Measure whether verified sessions reduce daily screen time compared with an unverified timer.

# 14 Conclusion

A phone cannot prove that you were focused. It can prove something narrower and still useful: that it lay face-down and still, that it stayed in one app, and that something physical answered it at moments it chose. The server checks all of this on its own clock and records it once on a public ledger. Our measurements show where that proof holds and where it does not. It stopped every scripted UI session we ran on real hardware. It does not stop a device that lies about its own sensors. The economics avoid the two failure modes of earlier designs: forfeits that enrich the operator, and emissions that collapse. FACEDOWN is the first consumer of

PRESENCE. The broader product is proof that a human performed a process, which every airdrop, quest and reward program currently lacks.

# References


[1] N. Castelo, K. Kushlev, A. F. Ward, M. Esterman, et al. Blocking mobile internet on smartphones improves sustained attention, mental health, and subjective well-being. *PNAS Nexus* 4(2):pgaf017, 2025. doi:10.1093/pnasnexus/pgaf017.

[2] S. D. Halpern, B. French, D. S. Small, K. Saulsgiver, et al. Randomized trial of four financial-incentive programs for smoking cessation. *N Engl J Med* 372:2108–2117, 2015. doi:10.1056/NEJMoa1414293.

[3] A. Sachdeva, J. Somasundaram, K. K. Sharma. Self-selected goals outperform assigned goals in reducing mobile phone usage: evidence from a randomized controlled trial. *Soc Sci Med* 406:119683, 2026. doi:10.1016/j.socscimed.2026.119683.

[4] D. J. Grüning, F. Riedel, P. Lorenz-Spreen. Directing smartphone use through the self-nudge app one sec. *PNAS* 120:e2213114120, 2023. doi:10.1073/pnas.2213114120.

[5] J. T. Kullgren, A. B. Troxel, G. Loewenstein, D. A. Asch, et al. Individual- versus group-based financial incentives for weight loss: a randomized, controlled trial. *Ann Intern Med* 158:505–514, 2013. doi:10.7326/0003-4819-158-7-201304020-00002.

[6] L. Derksen, J. Kerwin, N. Ordaz Reynoso, O. Sterck. Appointments: a more effective commitment device for health behaviors. arXiv:2110.06876, 2021.

[7] Y. Liu, H. Huang. WiP: Characterizing and defending against mobile-agent-driven MFA automation. arXiv:2609.02154, 2026.

[8] I. Querejeta-Azurmendi, P. Papadopoulos, M. Varvello, A. Nappa, J. Zhang, B. Livshits. ZKSENSE: a friction-less privacy-preserving human attestation mechanism for mobile devices. arXiv:1911.07649, 2019.

[9] Y. Gao, F. Li, R. Shi, R. Cheng, J. Zhang, B. Han, S. Chen. NFT games: an empirical look into the play-to-earn model. arXiv:2602.13882, 2026.


---

*Every number in this paper was measured from the code, the test suite or the devnet database, or is quoted from a cited abstract. Figures 4 and 5 (right) are labelled models. Nothing here ran on mainnet; all tokens are SKR-TEST, a valueless test mint. To reproduce the figures and Tables 5 and 6:* `uv run -with matplotlib -with numpy python docs/whitepaper/make_figures.py`.